

Contents

1	Introduction	1
1.1	Prologue	1
1.2	Automation is everywhere	1
1.3	Contributions	1
1.3.1	Alternative type-class solver in <i>Coq-Elpi</i>	1
1.3.2	<i>Coq-Elpi</i> : Higher-order unification support	1
1.3.3	Static determinacy checking for <i>Elpi</i>	1
1.3.4	First-order static determinacy checker formalization	1
2	State of the Art	3
2.1	The <i>Rocq</i> proof assistant	3
2.1.1	The basis of the language	3
2.1.2	Elaboration	3
2.1.3	Ad-hoc polymorphism: Type classes	3
2.2	The <i>Elpi</i> language	4
2.2.1	λ -terms representation	4
2.2.2	Hints about unification	5
2.2.3	Controlling backtracking: the ! operator	5
2.2.4	<i>CHR</i> : goal suspension	5
2.3	<i>Coq-Elpi</i> : <i>Rocq</i> and <i>Elpi</i> interleaving	5
2.3.1	Concluding remarks	5
3	Alternative type-class solver in <i>Coq-Elpi</i>	7
3.1	Why typeclass resolution in <i>Elpi</i> ?	7
3.2	From <i>Rocq</i> goals to <i>Elpi</i> proofs	7
3.2.1	Rule encoding	7
3.2.2	Goal encoding	7
3.3	Alternative search indexing	7
3.3.1	Discrimination trees	7
3.3.2	Some ideas on tabling	7
3.4	Results and discussion	7
4	Higher-order unification support	9
4.1	η -equivalent terms	9
4.2	β -equivalent terms	9
4.3	Practical evaluation	9

5	Determinacy checking for <i>Elpi</i>	11
5.1	Why and what is determinacy?	11
5.2	Rule checking	11
5.3	Mutual exclusion	11
5.4	Caveats	11
5.5	Experience report	11
6	First-order static determinacy checker formalization	13
6.1	The language	13
6.2	The tree semantics	13
6.3	The stack semantics	13
6.4	Semantics equivalence	13
6.5	The main theorem	13
7	Conclusion and Perspectives	15
	Bibliography	17

CHAPTER 1

Introduction

1.1 Prologue

Automation everywhere in CS, we want to set up boilerplates to help our development.

Examples:

- Continuous integration
- Macros are everywhere
- Artificial intelligence

We delegate tasks to some tools.

expert system →

example plus frappant:

artificial intelligence: based on probability but not reproducible, not predicatable

several other examples in computer sciences:

...

1.2 Automation is everywhere

In the domain of formal proof automation is everywhere, and it is formally called elaboration [51]

In formal proof, proofs are verbose and have lot of details: types, coercions, canonical structures, type-classes.

In theorem provers the user benefit of notations, implicit inference and tactics to ease his job.

Simply typed lambda calculus is an example of automation: an algo checks if ...

1.3 Contributions

In this manuscript we focus on an alternative type-class solver written in the elpi meta-language.

We also talk about a formalized tool...

1.3.1 Alternative type-class solver in *Coq-Elpi*

1.3.2 *Coq-Elpi*: Higher-order unification support

1.3.3 Static determinacy checking for *Elpi*

1.3.4 First-order static determinacy checker formalization

CHAPTER 2

State of the Art

At the very base of my PhD there are two softwares, the Rocq Proof assistant (formerly known as *Coq* [6]) and the *Elpi* [13].

2.1 The *Rocq* proof assistant

[6]

Theorem provers with HO logic programming languages: [15]

Mathematical components: [32]

Abella: [21]

SSReflect: [22]

Isabelle: [38]

2.1.1 The basis of the language

Function vs Propositions

2.1.2 Elaboration

In fig. 2.1 (taken from [51]), we have a skemtical representation of the elaboration process. [51]

Coercions: [4]

2.1.3 Ad-hoc polymorphism: Type classes

What is a method?

Wadler: [53]

Matthieu: [47]

Haskell: [54]

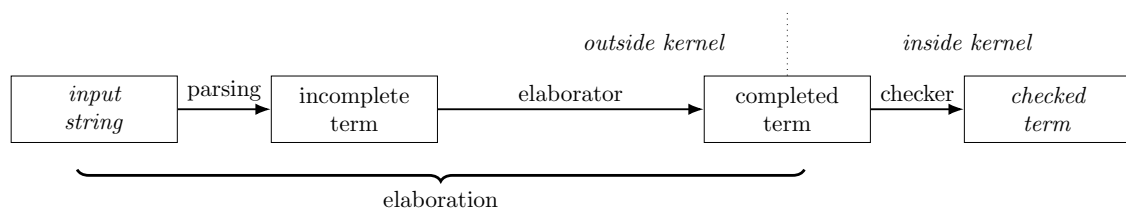


Figure 2.1: Elaboration skeme

```

Class Eqb (T : Type) := {eqb : T -> T -> bool}.

Instance eqNat : Eqb nat := {eqb a b := Nat.eqb a b}.
Instance eqProd (A: Type) (B: Type) : Eqb A -> Eqb B -> Eqb (A * B) :=
  {eqb a b := eqb a.1 b.1 && eqb a.2 b.2}.

```

(a) Eqb typeclass in Rocq

```

class Eqb a where eqb :: a -> a -> Bool

instance Eqb Integer where eqb = integerEq
instance (Eqb a, Eqb b) => Eqb (a, b) where
  eqb (x1, y1) (x2, y2) = eqb x1 x2 && eqb y1 y2

```

(b) Eqb typeclass in Haskell

Figure 2.2: Eqb typeclass in different languages

ATTENZIONE: Eqb is not eqb

We can define a type class for testing equality. In fig. 2.2, we give two implementations of Eqb in the Rocq and Haskell languages. The similarity between the two pieces of code is striking, but in Rocq we can go further. Type classes can be used not only for function overloading, but also for theorem overloading. In coq a class can in fact not also carry function specifications, but also the properties that these functions should satisfy. For example, we could have a method, say *eqP* certifying that for each implementation of *eqb* there is a proof that the two objects are structurally equivalent. This would also mean that each implementation of *Eqb* should implement *eqP*.

This last implementation with *eqb* and *eqP* is the one used by *ssreflect* [22] through [9]

Type classes are deeply used in libraries such as *stdpp*, *iris*, ...

Cite Haskell [25], in caml it does not exist: `+. vs +`

IRIS: [29]

2.2 The *Elpi* language

Elpi [13] is a dialect of λ prolog [34, 37]

Teyjus: [36, 35]

Elf: [39]

Twelf: [41]

Lambda prolog: [44]

Mercury: [46]

WAM: [3]

Prolog semantics: [11]

2.2.1 λ -terms representation

HOAS: [40]

2.2.2 Hints about unification

Miller: [34]

Huet: [28]

Modes: [55]

2.2.3 Controlling backtracking: the ! operator

SWI prolog: [48]

Andrews: [2]

Vink: [10]

2.2.4 CHR: goal suspension

[20, 19]

CHR elpi: [24, section 4.3]

CHR Pfenning: [33]

2.3 *Coq-Elpi: Rocq and Elpi* interleaving

Coqelpi: [49]

Hierarchy Builder: [9]

Trackt: [7]

Equality test [23]

Derive: [50]

2.3.1 Concluding remarks

Overall, Elpi is good for elaboration

CHAPTER 3

Alternative type-class solver in *Coq-Elpi*

Typeclasses are an elegant way to implement ad-hoc polymorphism [52], so that, the same symbol can be reused in different context while keeping the same mathematical properties. From an implementation point of view, typeclasses are

3.1 Why typeclass resolution in *Elpi*?

[16]

Rule base automation!

3.2 From *Rocq* goals to *Elpi* proofs

3.2.1 Rule encoding

The hooks

3.2.2 Goal encoding

3.3 Alternative search indexing

3.3.1 Discrimination trees

Discrimination trees: Pfenning notes *

TODO: do some benchmarks

Discrimination trees: opt for lists and tree depths.

3.3.2 Some ideas on tabling

[45, 42]

3.4 Results and discussion

*<https://www.cs.cmu.edu/~fp/courses/99-atp/lectures/lecture28.html>

Higher-order unification support

[17]

?: [14]

Variadic functions are typechecked in coq [8]

4.1 η -equivalent terms

4.2 β -equivalent terms

[1]

Beta problema outside the pattern can come back in LL [36]

4.3 Practical evaluation

IRIS: [29]

CHAPTER 5

Determinacy checking for *Elpi*

5.1 Why and what is determinacy?

[12, 46, 26, 27, 31, 30]

5.2 Rule checking

5.3 Mutual exclusion

Using discrimination tree

5.4 Caveats

For example, mutual exclusion is not perfect: over approximation of unification
Rule checking ignores flexible rule loading

5.5 Experience report

CHAPTER 6

First-order static determinacy checker formalization

Using `ssreflect` [\[22\]](#)

6.1 The language

Input must come before outputs

Inputs must come before outputs, with Disc tree this is not mandatory, since unification is weaker.

EG:

```
f 3 3 (g X X).  
f 3 4 (g 3 4).  
  
main:-  
  f X Y (g X Y).
```

In the code above, both rules can be used with the query, but, the discrimination tree, save the world

6.2 The tree semantics

6.3 The stack semantics

[\[43\]](#)

6.4 Semantics equivalence

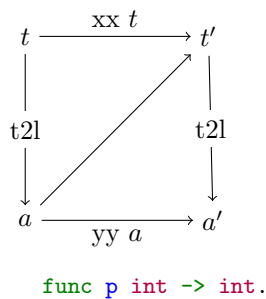
[\[5\]](#)

6.5 The main theorem

CHAPTER 7

Conclusion and Perspectives

[18]



Bibliography

- [1] Andreas Abel and Brigitte Pientka. Higher-order dynamic pattern unification for dependent types and records. In Luke Ong, editor, *Typed Lambda Calculi and Applications*, pages 10–26, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [2] James H. Andrews. The witness properties and the semantics of the prolog cut. *Theory Pract. Log. Program.*, 3(1):1–59, January 2003. doi:[10.1017/S1471068402001540](https://doi.org/10.1017/S1471068402001540).
- [3] Hassan Aït-Kaci. *Warren’s Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:[10.7551/mitpress/7160.001.0001](https://doi.org/10.7551/mitpress/7160.001.0001).
- [4] Gilles Barthe. Implicit coercions in type systems. In Stefano Berardi and Mario Coppo, editors, *Types for Proofs and Programs*, pages 1–15, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
- [5] Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- [6] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer Verlag, 2004.
- [7] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:[10.1145/3573105.3575676](https://doi.org/10.1145/3573105.3575676).
- [8] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013.
- [9] Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPICs*, pages 34:1–34:21, 2020. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.FSCD.2020.34>, doi:[10.4230/LIPICs.FSCD.2020.34](https://doi.org/10.4230/LIPICs.FSCD.2020.34).
- [10] A. de Bruin and E. P. de Vink. Continuation semantics for prolog with cut. In Josep Díaz and Fernando Orejas, editors, *TAPSOFT ’89*, pages 178–192. Springer, 1989.
- [11] Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:[Debray1988](https://doi.org/Debray1988).
- [12] Saumya K. Debray and David S. Warren. Functional computations in logic programs. *ACM Trans. Program. Lang. Syst.*, 11(3):451–481, July 1989. doi:[10.1145/65979.65984](https://doi.org/10.1145/65979.65984).

- [13] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λ Prolog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/978-3-662-48899-7_32.
- [14] Amy Felty. Encoding the calculus of constructions in a higher-order logic. In M. Vardi, editor, *lics93*, pages 233–244. IEEE, June 1993. doi:10.1109/LICS.1993.287584.
- [15] Amy Felty and Dale Miller. Specifying theorem provers in a higher-order logic programming language. In Ewing Lusk and Ross Overbeek, editors, *9th International Conference on Automated Deduction*, pages 61–80, Berlin, Heidelberg, 1988. Springer Berlin Heidelberg.
- [16] Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- [17] Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.
- [18] Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- [19] Thom Frühwirth. Constraint handling rules - what else? In Nick Bassiliades, Georg Gottlob, Fariba Sadri, Adrian Paschke, and Dumitru Roman, editors, *Rule Technologies: Foundations, Tools, and Applications*, pages 13–34, Cham, 2015. Springer International Publishing.
- [20] Thom Frühwirth. Theory and practice of constraint handling rules. *The Journal of Logic Programming*, 37(1):95–138, 1998. URL: <https://www.sciencedirect.com/science/article/pii/S0743106698100055>, doi:10.1016/S0743-1066(98)10005-5.
- [21] Andrew Gacek. The abella interactive theorem prover (system description). In *Proceedings of the 4th International Joint Conference on Automated Reasoning, IJCAR '08*, page 154–161, Berlin, Heidelberg, 2008. Springer-Verlag. doi:10.1007/978-3-540-71070-7_13.
- [22] Georges Gonthier and Enrico Tassi. A language of patterns for subterm selection. In Lennart Berlinger and Amy Felty, editors, *Interactive Theorem Proving*, pages 361–376, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [23] Benjamin Grégoire, Jean-Christophe L  chenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.
- [24] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. *Mathematical Structures in Computer Science*, 29:1–26, 03 2019. doi:10.1017/S0960129518000427.

- [25] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip L. Wadler. Type classes in haskell. *ACM Trans. Program. Lang. Syst.*, 18:109–138, March 1996. doi:[10.1145/227699.227700](https://doi.org/10.1145/227699.227700).
- [26] Michael Hanus and Kai-Oliver Prott. Determinism Types for Functional Logic Programming. In *Proceedings of PPDP*, pages 47–59, 2025.
- [27] Fergus Henderson, Zoltan Somogyi, and Thomas Conway. Determinism analysis in the mercury compiler. In *Proceedings of Australian Computer Science Conference*, pages 337–346, 08 1996.
- [28] G.P. Huet. A unification algorithm for typed λ -calculus. *Theoretical Computer Science*, 1(1):27–57, 1975. URL: <https://www.sciencedirect.com/science/article/pii/0304397575900110>, doi:[10.1016/0304-3975\(75\)90011-0](https://doi.org/10.1016/0304-3975(75)90011-0).
- [29] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi:[10.1017/S0956796818000151](https://doi.org/10.1017/S0956796818000151).
- [30] Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. *Theory and Practice of Logic Programming*, 11(4-5):537–553, 2011.
- [31] Lunjin Lu and Andy King. Determinacy inference for logic programs. *Lecture Notes in Computer Science*, 3444:108–123, 04 2005. doi:[10.1007/978-3-540-31987-0_9](https://doi.org/10.1007/978-3-540-31987-0_9).
- [32] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, September 2022. doi:[10.5281/zenodo.7118596](https://doi.org/10.5281/zenodo.7118596).
- [33] Spiro Michaylov and Frank Pfenning. Higher-order logic programming as constraint logic programming. In *Position Papers for the First Workshop on Principles and Practice of Constraint Programming*, pages 221–229, Newport, Rhode Island, April 1993. Brown University.
- [34] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- [35] Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, New York, NY, USA, 2012.
- [36] Gopalan Nadathur. The metalanguage λ prolog and its implementation. In Herbert Kuchen and Kazunori Ueda, editors, *Functional and Logic Programming*, pages 1–20, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.
- [37] Gopalan Nadathur and Dale A. Miller. An overview of lambda-prolog. In *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, pages 810–827, Seattle, WA, USA, 1988. MIT Press.
- [38] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.

- [39] Frank Pfenning. Elf: a language for logic definition and verified metaprogramming. In *Proceedings of the Fourth Annual IEEE Symposium on Logic in Computer Science (LICS 1989)*, pages 313–322. IEEE Computer Society Press, June 1989.
- [40] Frank. Pfenning and Conal Elliott. Higher-order abstract syntax. In *Proceedings of PLDI*, page 199–208. Association for Computing Machinery, 1988. doi:10.1145/53990.54010.
- [41] Frank Pfenning and Carsten Schürmann. System description: Twelf — a meta-logical framework for deductive systems. In *Automated Deduction — CADE-16*, pages 202–206, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [42] Brigitte Pientka. Tabling for higher-order logic programming. In Robert Nieuwenhuis, editor, *Automated Deduction – CADE-20*, pages 54–68, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- [43] Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors, *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.
- [44] Xiaochu Qi. *An Implementation of the Language Lambda Prolog Organized around Higher-Order Pattern Unification*. Ph.d. thesis, University of Minnesota, Minneapolis, MN, USA, 2009. Available at the University Digital Conservancy.
- [45] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. Tabled typeclass resolution. *CoRR*, abs/2001.04301, 2020. URL: <https://arxiv.org/abs/2001.04301>, arXiv:2001.04301.
- [46] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1):17–64, 1996. High-Performance Implementations of Logic Programming Systems. URL: <https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684>, doi:10.1016/S0743-1066(96)00068-4.
- [47] Matthieu Sozeau and Nicolas Oury. First-class type classes. In *Proceedings of TPHOLs*, volume 5170 of *LNCS*, pages 278–293. Springer, 2008.
- [48] SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-Prolog PlDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.
- [49] Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: <https://inria.hal.science/hal-01637063>.
- [50] Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPICs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>, doi:10.4230/LIPICs.CVIT.2016.23.

- [51] Enrico Tassi and Freek Wiedijk. ? In ?, editor, ?, 2026.
- [52] Philip Wadler. Theorems for free! In *Proceedings of FPCA*, FPCA '89, page 347–359. Association for Computing Machinery, 1989. doi:10.1145/99370.99404.
- [53] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/75277.75283.
- [54] Markus Wenzel. Type classes and overloading in higher-order logic. In Elsa L. Gunter and Amy Felty, editors, *Theorem Proving in Higher Order Logics*, pages 307–322, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.
- [55] Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*, 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.

Tritre francais

Davide FISSORE